

ESCUELA DE
INGENIERÍA INFORMÁTICA



PONTIFICIA
UNIVERSIDAD
CATÓLICA DE
VALPARAÍSO

REPORTE PROYECTO

Manejo de sistema de salud

Martín Lira
Amanda Letelier
Benjamín González

Septiembre 2026

ÍNDICE GENERAL

1. SIA 1 - Análisis de los datos y funcionalidades.....	3
1.1. Análisis de datos del sistema:.....	3
1.2. Funcionalidades del sistema.....	4
2. SIA 2 - Diseño conceptual en UML.....	4
3. SIA 3 - Cumplimiento de buenas prácticas.....	5
3.1. Atributos privados en todas la clases, incluyendo getters y setters.....	5
4. SIA 4 - Implementación de 2 colecciones de objetos, una de ellas debe ser un mapa, ambas deben pertenecer al JCF.....	5
4.1. Implementación colección “Área”	5
4.2. Implementación colección “Cama”	5
4.3. Implementación colección “Paciente”	6
5. SIA 5 - Implementación de 2 clases que utilicen sobrecarga de métodos.....	6
5.1. Clase área:.....	6
5.2. Clase hospital:.....	6
6. SIA 6 - Implementación de 2 clases que utilicen sobrescritura de métodos.....	6
6.1. Clase Adulto:.....	6
6.2. Clase Nino:.....	6
7. SIA 7 - Menú del sistema con funcionalidades.....	6
7.1. SIA 7.1 - Inserción manual (agregar elemento).....	6
7.2. SIA 7.2 - Mostrar listado de elementos.....	7
8. SIA 8 - Menú del sistema con funcionalidades.....	7
8.1. SIA 8.1 - Edición de elemento (Modificar elemento).....	7
8.2. SIA 8.2 - Eliminación de un elemento.....	7
8.3. SIA 8.3 Búsqueda de un elemento.....	8
9. SIA 9 - Implementación de funcionalidades propias.....	8
9.1. Reubicar a pacientes por gravedad.....	8
9.2. Automatización de altas.....	8
10. SIA 10 - Implementación de funcionalidades en ventana y consolas.....	9
11. SIA 11 - Persistencia de datos.....	9
12. SIA 12 - Implementación de 2 excepciones en el programa.....	9
12.1. ExcepcionAreaNoExistente.....	9
12.2. ExcepcionPacienteNoEncontrado.....	10
13. SIA 13 - Uso de github.....	10

1. SIA 1 - Análisis de los datos y funcionalidades.

1.1. Análisis de datos del sistema:

El sistema de información administra los servicios de salud de un hospital. Para que esto funcione de manera eficiente separamos por bloques según cada requerimiento de un hospital.

Datos de áreas clínicas: Es importante definir sectores dependiendo de la gravedad o situación de cada paciente, es por eso que cada área tiene un **código** que funciona como identificador único, un **nombre** que detalla la especialidad de la zona, un aforo de **capacidad máxima**, además un listado de todas las **camas** asignadas en el área.

Datos de hospital: Esta clase representa a la institución en su totalidad, en la cual tenemos registros del **nombre** y **dirección** del hospital, y también de todas las **áreas** que lo componen. Además de un listado dinámico de los **pacientes sin asignar**, lo que permite mantener el sistema actualizado respecto a las personas que están por ingresar.

Datos de camas: Para una correcta atención de pacientes, se necesita saber exactamente qué camas hay disponibles, por esa razón es que cada una tiene un **id** único, un número que indica la **disponibilidad** (0: ocupada, 1: disponible, 2: en limpieza, 3: en mantenimiento), el nombre de la **categoría de cama**, y en caso de que la cama esté en uso, se enlaza con los datos del **paciente actual**.

Datos de pacientes: Al registrar a cada paciente podemos obtener datos de identificación como su **nombre**, **rut** y **edad**, este último luego nos permite clasificar como *adulto* o *niño* al paciente. Además, también obtenemos los **signos vitales** de la persona, ligado a esto tenemos la **gravedad**, la cual nos sirve para derivar al paciente un área dependiendo del número que dé como resultado. Por último tenemos el registro de la **cama asignada** al paciente.

Datos de adulto: Esta clase es una extensión de paciente, por lo que además de heredar los datos de **identificación** y **salud**, podemos registrar el **sistema de previsión** al que pertenece.

Datos de niño: Extensión de clase paciente, además de heredar sus **datos personales** y de **salud**, se deben registrar los datos personales del apoderado, como **nombre** y **rut**.

Datos de signos vitales: Es fundamental registrar los signos vitales para asignar la situación clínica y luego derivar al paciente, por ello se registran datos como la **frecuencia cardíaca**, la **presión arterial**, la **saturación** y por último la **temperatura** de la persona.

1.2. Funcionalidades del sistema

Para dar sentido y utilidad al sistema de información, este debe resolver las necesidades administrativas dentro de este tipo de institución. Es por ello que en nuestro proyecto se implementaron un conjunto de funcionalidades diseñadas para optimizar el flujo clínico, siendo las siguiente:

Gestión de inventario y departamentos: Las clases **Area** y **Cama** permiten registrar, asignar y estructurar los distintos sectores del hospital. Es posible controlar el aforo mediante el uso de *capacidadMaxima* de cada sector, además de mantener un conteo actualizado de las camas asignadas a cada área, diferenciando si están disponibles, no disponibles, en limpieza o, en su defecto, en mantenimiento.

Control de ingreso y asignación de pacientes: De esta manera, se optimiza el ingreso de pacientes al sistema del hospital, determinando el tipo de atención (pediátrica o adulto) para cada persona a partir de los datos personales entregados. También es posible derivarlos directamente a una área según su nivel de gravedad, el cual es calculado a partir de los datos médicos entregados (signosVitales).

Flexibilidad y motores de búsqueda: La implementación de la sobrecarga de métodos permite distintas formas de encontrar información; ya sea localizar una cama mediante su identificador, o por el rut del paciente que se encuentra usándola. También permite la búsqueda de un área clínica del hospital, lo cual es posible tanto por su identificación única (id) o bien, por su nombre.

2. SIA 2 - Diseño conceptual en UML

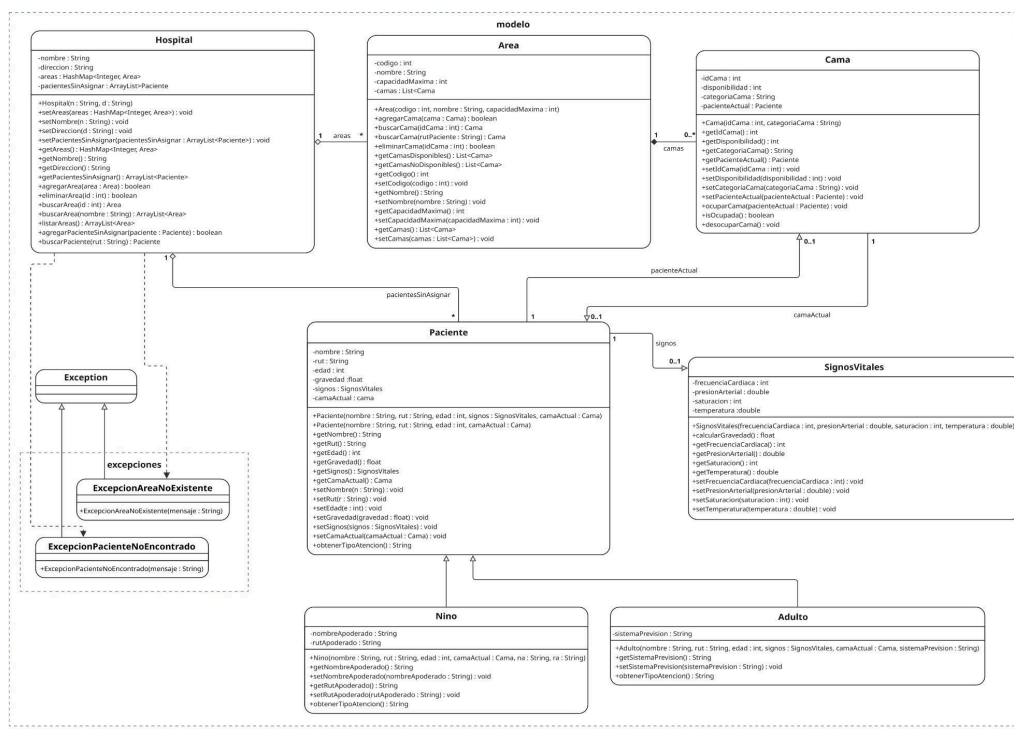


Diagrama 1: Modelos y excepciones

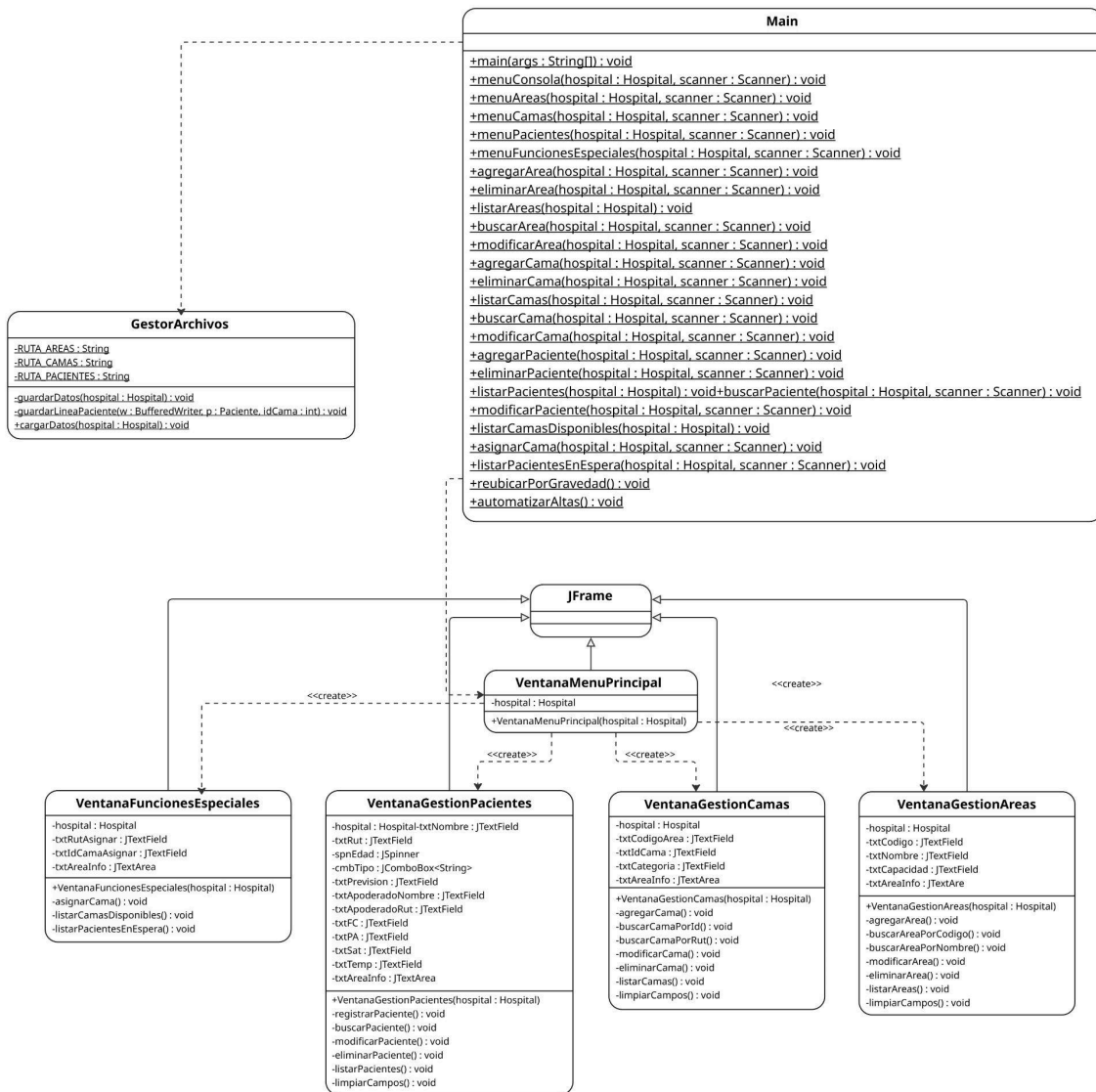


Diagrama 2: Ventanas y main

Se puede visualizar el diseño UML a mayor detalle en el siguiente [link](#).

El sistema está modelado siguiendo una separación entre la lógica de negocio y su capa gráfica. Se ven representados mediante 2 diagramas UML, en donde el primero representa las clases de dominio, persistencia de datos y las respectivas excepciones, mientras que en el segundo se modela el Main (Modo consola) y las distintas ventanas desarrolladas mediante Java Swing.

3. SIA 3 - Cumplimiento de buenas prácticas

3.1. Atributos privados en todas la clases, incluyendo getters y setters

Cada clase cumple con tener cada atributo privado para asegurar un acceso controlado a todos los datos, garantizando de esta manera el encapsulamiento y evitando

modificaciones no deseadas. Además es posible la lectura y modificación de los datos mediante sus respectivos métodos getters y setters.

4. SIA 4 - Implementación de 2 colecciones de objetos, una de ellas debe ser un mapa, ambas deben pertenecer al JCF

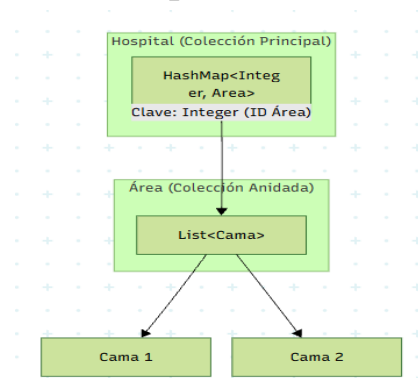


Diagrama representativo del anidamiento de colecciones

4.1. Implementación colección “Área”

En la clase *Hospital*, se implementa un *HashMap* como colección principal, cumpliendo con la exigencia de utilizar un mapa para optimizar las búsquedas.

4.2. Implementación colección “Cama”

A partir de la colección mencionada en el punto anterior, se implementó una colección de manera anidada, donde cada objeto de la clase *Area* contendrá de manera interna una propia colección de tipo *List<Cama>* como *ArrayList*, esto para almacenar las distintas camas pertenecientes a un sector determinado del servicio de salud, de esta manera obteniendo una correcta organización y gestión.

4.3. Implementación colección “Paciente”

De manera adicional, la clase *Hospital* cuenta con un *ArrayList<Paciente>* en el atributo *pacientesSinAsignar*, utilizado como lista auxiliar para los pacientes en espera de ser asignados a una cama.

5. SIA 5 - Implementación de 2 clases que utilicen sobrecarga de métodos.

5.1. Clase área:

Se implementa la sobrecarga en el método **buscarCama**. Se definen dos versiones, permitiendo encontrar una cama consultando por datos dependiendo del contexto; la primera versión recibe el identificador único de la cama (*int idCama*), mientras que la segunda recibe el rut del paciente asignado a la cama (*String rutPaciente*)

5.2. Clase hospital:

Se implementa la sobrecarga del método **buscarArea**, en la cual se definen dos versiones dependiendo de la información disponible. La primera recibe el identificador único del área (*int id*), mientras que en la segunda recibe el nombre del

sector (*String nombre*), retornando una lista con todas las áreas que coincidan con dicho parámetro.

6. SIA 6 - Implementación de 2 clases que utilicen sobreescritura de métodos.

6.1. Clase Adulto:

Se implementa una sobreescritura al método heredado **obtenerTipoAtencion**, permitiendo modificar el comportamiento que tiene por defecto la clase Paciente (retorna “Atención general.”) para así poder adaptarlo a su debido contexto, especificando que el paciente requiere “Atención de adultos.”

6.2. Clase Nino:

Siguiendo el mismo principio, en esta clase se sobreescribe el método **obtenerTipoAtencion**, adaptando la respuesta al propio contexto del paciente, por lo que retorna “Atención pediátrica.”

7. SIA 7 - Menú del sistema con funcionalidades

Para cumplir con la exigencia de agregar y listar elementos de las colecciones, se implementaron métodos para cada clase:

7.1. SIA 7.1 - Inserción manual (agregar elemento).

agregarArea: El sistema solicita por consola el código, nombre y capacidad máxima para instanciar un nuevo objeto Area, luego se intenta añadir esta nueva área a la colección del hospital mediante `hospital.agregarArea()`. Si la inserción es exitosa, se confirma al usuario. Si el identificador ya existe, el sistema bloquea la acción y muestra un error, evitando duplicidad de códigos.

agregarPaciente: El sistema solicita los datos personales del paciente (nombre, rut y edad) junto con sus datos médicos (frecuencia cardiaca, presión, saturación y temperatura) para crear un objeto `signosVitales`. A partir de la edad ingresada, el programa asigna un objeto correspondiente; si es menor de 18 años, se instancia un `Nino` (solicitando datos del apoderado); de lo contrario, instancia un `Adulto` (solicitando su previsión).

7.2. SIA 7.2 - Mostrar listado de elementos.

listarAreas: El sistema solicita la colección de áreas del hospital y, tras validar que existan registros, recorre la lista para imprimir por consola el código, el nombre y la capacidad máxima de cada unidad clínica.

listarPacientes: El sistema despliega un reporte integral de todos los pacientes registrados, iterando primero sobre la lista de espera (pacientes sin cama) y luego recorriendo todas las áreas y camas del hospital para identificar a los pacientes asignados. Durante este recorrido, se extrae e imprime por consola la información

clínica y personal de cada individuo, junto con su estado actual o ubicación específica en el recinto, notificando al usuario en caso de que el sistema no contenga registros activos.

8. SIA 8 - Menú del sistema con funcionalidades

8.1. SIA 8.1 - Edición de elemento (Modificar elemento).

modificarArea: El sistema solicita el código identificador y, tras localizar el área mediante `hospital.buscarArea()`, despliega un submenú que permite al usuario actualizar de forma independiente el nombre de la unidad o ajustar su capacidad máxima operativa.

modificarPaciente: Utilizando `rut` como clave de búsqueda en `hospital.buscarPaciente()`, el sistema recupera el registro y ofrece modificar atributos básicos o actualizar integralmente el cuadro clínico. Si se elige modificar los signos vitales, se requiere el ingreso de nuevas métricas, lo que recalcula e imprime automáticamente el nuevo indicador de gravedad del paciente.

8.2. SIA 8.2 - Eliminación de un elemento.

eliminarArea: El sistema solicita el código de la unidad a remover y ejecuta el método `hospital.eliminarArea()`. Esta operación está protegida por un bloque try-catch que gestiona excepciones en caso de que el área no exista, asegurando la continuidad del programa y notificando al usuario del resultado.

eliminarPaciente: Tras solicitar el `rut`, el sistema implementa una búsqueda iterativa en dos niveles. Primero, recorre la lista de pacientes sin asignar; si hay coincidencia, remueve el registro directamente. Si no se encuentra, itera a través de todas las áreas y sus respectivas camas; al hallar al paciente internado, ejecuta el método `desocuparCama()`, liberando el recurso físico y eliminando al paciente del sistema en un solo paso.

8.3. SIA 8.3 Búsqueda de un elemento.

buscarArea: El sistema solicita el código del área y utiliza el método `buscarArea(codigo)` del objeto `hospital` para localizarla. Si la encuentra, despliega el nombre y la capacidad máxima actual; de lo contrario, el manejo de excepciones (`catch (ExcepcionAreaNoExistente e)`) captura el error y muestra el mensaje correspondiente, evitando caídas del programa.

buscarPaciente: El usuario ingresa el `rut` del paciente a buscar. El sistema invoca `buscarPaciente` y, al encontrar coincidencias, presenta en pantalla el nombre, `rut` y edad del paciente. Al igual que en las áreas, la robustez se garantiza mediante un bloque try-catch que gestiona la excepción `ExcepcionPacienteNoEncontrado`.

9. SIA 9 - Implementación de funcionalidades propias.

El sistema contempla mecanismos eficientes para el control de camas e ingreso de pacientes, asegurando que los recursos del hospital se administren de manera coherente. Esto es posible principalmente mediante dos funcionalidades.

9.1. Reubicar a pacientes por gravedad

Esta funcionalidad es parte de las clases **Main** (para modo consola) y en **VentanaFuncionesEspeciales**, asimismo corresponde al método **reubicarPorGravedad**. Nos permite asegurar que los pacientes más críticos reciban atención oportuna en unidades de mayor complejidad según la gravedad. El método itera sobre las áreas del hospital, identifica a los pacientes con una gravedad severa y los traslada automáticamente a camas disponibles en áreas como UCI o urgencias. Esta funcionalidad cumple con operar sobre un subconjunto filtrado por criterio, procesando específicamente a los pacientes cuya gravedad sea igual o superior a 4.0, siempre y cuando estos no se encuentren ya previamente en áreas de complejidad.

9.2. Automatización de altas

Esta funcionalidad es parte de las clases **Main** (para modo consola) y en **VentanaFuncionesEspeciales**, asimismo corresponde al método **automatizarAltas**. Esto nos permite gestionar de manera eficiente la liberación de camas evaluando la recuperación de los pacientes que estuvieron internados. El método itera sobre la lista completa de áreas y sus respectivas camas ocupadas, por lo que si identifica a un paciente con una gravedad menor o igual a 1.0, le otorga el alta médica, desvinculándolo de la cama y cambiando el estado de esta, así quedando disponible para nuevas hospitalizaciones. Esta funcionalidad cumple con operar sobre un subconjunto filtrado por criterio, procesando únicamente a los pacientes cuya gravedad se encuentra bajo el umbral previamente mencionado.

10. SIA 10 - Implementación de funcionalidades en ventana y consolas.

Al iniciar la aplicación, el sistema solicita al usuario elegir entre la ejecución por consola y una Interfaz Gráfica de Usuario (GUI). La opción visual fue desarrollada utilizando Java Swing, organizando la navegación desde un menú central (**VentanaMenuPrincipal**) hacia módulos específicos:

Gestión de entidades: Ventanas independientes equipadas con formularios para administrar Áreas (**VentanaGestionAreas**), Camas (**VentanaGestionCamas**) y Pacientes (**VentanaGestionPacientes**).

Operaciones clínicas: Un módulo dedicado exclusivamente a la asignación de recursos y automatización de altas (**VentanaFuncionesEspeciales**).

La interfaz captura datos, previene errores utilizando alertas emergentes (**JOptionPane**) y despliega los resultados de todas las operaciones en paneles de texto

integrados (JTextArea), cumpliendo con la exigencia de operar la totalidad del sistema mediante ventanas.

11. SIA 11 - Persistencia de datos.

- 11.1. El sistema incorpora persistencia de datos mediante la clase GestorArchivos, utilizando tres archivos de texto (areas.txt, camas.txt, pacientes.txt) con un formato separado por comas.

Carga inicial: Al iniciar la aplicación, el método cargarDatos() lee los archivos para instanciar y reconstruir automáticamente las áreas, las camas y la ubicación exacta de cada paciente (internado o en espera).

Grabado al salir: Al finalizar la sesión, el método guardarDatos() recorre las colecciones actualizadas y sobrescribe los archivos de texto, asegurando la conservación del estado operativo del hospital para la próxima ejecución.

12. SIA 12 - Implementación de 2 excepciones en el programa.

Para asegurar la robustez del programa y así evitar cierres inesperados, fueron implementadas dos excepciones según el contexto del sistema. El manejo de ambas es mediante try-catch.

12.1. ExcepcionAreaNoExistente

Esta excepción se hereda de la clase base **Exception**. Despliega un mensaje de error cuando el sistema intenta localizar o interactuar con un sector del hospital que no existe en los registros.

12.2. ExcepcionPacienteNoEncontrado

Siguiendo la misma estructura, esta clase también es heredada de la clase **Exception**. Fue implementada para casos en los que no sea posible encontrar a un paciente dentro de los datos del hospital, desplegando una alerta al no obtener resultados válidos.

13. SIA 13 - Uso de github

El desarrollo del proyecto fue gestionado a partir de un repositorio mediante la plataforma GitHub, el cual es accesible a través de este [link](#).